

Python Ala-2: Recursive Function

1. What is a Recursive Function?

Definition:

A recursive function is a function that calls itself repeatedly to solve a problem by breaking it into smaller sub problems until a base condition is reached.

Syntax:

```
def function_name(parameters):  
    if base_condition:  
        return value  
    else:  
        return function_name(smaller_value)
```

Example:

```
def print_num(n):  
    if n == 0:  
        return  
    print(n)  
    print_num(n-1)
```

```
print_num(5)
```

Output:

```
5  
4  
3  
2  
1
```

2. What is Base Case in Recursive Function?

Definition:

The base case is the stopping condition in a recursive function that prevents infinite recursion.

Syntax:

```
if condition:  
    return value
```

Example:

```
def show(n):  
    if n == 1:  
        return  
    print(n)  
    show(n-1)
```

```
show(5)
```

Base case: `n == 1`

3. What is Recursive Case?

Definition:

Recursive case is the part of the function where the function calls itself with a smaller input.

Syntax:

```
function_name(n-1)
```

Example:

```
def count(n):  
    if n == 0:  
        return  
    count(n-1)  
    print(n)
```

```
count(5)
```

Output:

```
1  
2  
3  
4  
5
```

4. Write a recursive function to find factorial of a number.

Definition:

Factorial of a number is the product of all positive integers less than or equal to that number.

Syntax:

```
def factorial(n):  
    if n == 0:  
        return 1
```

```
else:
    return n * factorial(n-1)
```

Example:

```
print(factorial(5))
```

Output:

120

5. Write a recursive function to find sum of first n numbers.

Definition:

Sum of first n numbers means adding numbers from 1 to n using recursion.

Syntax:

```
def sum_n(n):
    if n == 0:
        return 0
    return n + sum_n(n-1)
```

Example:

```
print(sum_n(5))
```

Output:

15

6. Write a recursive function to print numbers from 1 to n.

Definition:

This function prints numbers from 1 to n using recursive calls.

Syntax:

```
def print_numbers(n):
    if n == 0:
        return
    print_numbers(n-1)
    print(n)
```

Example:

```
print_numbers(5)
```

Output:

```
1
2
3
4
5
```

7. What happens if base case is missing in recursion?

Definition:

If a recursive function does not have a base case, it will run infinitely and cause a recursion error.

Example:

```
def loop(n):
    print(n)
    loop(n+1)
```

```
loop(1)
```

Output:

```
RecursionError: maximum recursion depth exceeded
```

8. Difference between Loop and Recursion

Definition:

Loop repeats a block of code using iteration, while recursion repeats using function calls.

Loop	Recursion
Uses for/while	Function calls itself
Less memory	More memory
Faster	Slower
Simple tasks	Complex problems

Example:

Loop:

```
for i in range(5):  
    print(i)
```

Recursion:

```
def show(n):  
    if n == 0:  
        return  
    print(n)  
    show(n-1)
```

9. Write a recursive function to find Fibonacci series.

Definition:

Fibonacci series is a sequence where each number is the sum of the previous two numbers.

Syntax:

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n-1) + fib(n-2)
```

Example:

```
for i in range(6):  
    print(fib(i))
```

Output:

```
0  
1  
1  
2  
3  
5
```

10. What are advantages of Recursive Function?

Definition:

Recursive functions simplify complex problems by dividing them into smaller parts.

Advantages:

- Simple and clean code
- Easy to understand
- Useful for trees and graphs
- Reduces long loops

Example:

```
def power(a, b):  
    if b == 0:  
        return 1  
    return a * power(a, b-1)  
  
print(power(2,3))
```

Output:

8